

Reviewer Pack — Hilbert Function Leading Coefficient Distinguishes Read-Twic...

Entry 4febd18902d4 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 06:21:18 UTC

Hilbert Function Leading Coefficient Distinguishes Read-Twice from Read-Once BPs

Entry ID: 4febd18902d4 Verdict: INCONCLUSIVE Recorded: 2026-05-09 06:21:18 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry **Field B** (complexity object): Read-Twice vs Read-Once Branching Programs

Statement:

For a read-twice BP P over n variables, the leading coefficient of the Hilbert function of the ideal I_P generated by its computation paths satisfies $\deg(I_P) = \Omega(n)$, while for read-once BPs, $\deg(I_P) = O(\log n)$.

Rationale (proposer’s reasoning):

The Hilbert function’s growth rate reflects the dimension of the ideal’s variety, which encodes the BP’s computational structure. Read-twice BPs generate higher-dimensional ideals due to repeated variable access, creating a detectable algebraic signature.

Taxonomy category: GCT (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 9386310757721b77

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_bp(n, read_twice):
        bp = []
        for _ in range(n):
            if read_twice:
                bp.append(random.choice(['x', 'y']))
                bp.append(random.choice(['x', 'y']))
            else:
                bp.append(random.choice(['x', 'y']))
        return bp

    def compute_monomials(bp):
        monomials = set()
        variables = {'x': 0, 'y': 0}
        for op in bp:
            if op == 'x':
                variables['x'] += 1
            elif op == 'y':
                variables['y'] += 1
            monomials.add(tuple(variables.copy()))
        return monomials

    def hilbert_function(monomials):
        max_degree = 0
        for monomial in monomials:
            degree = sum(monomial)
            if degree > max_degree:
                max_degree = degree
        return max_degree

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        read_twice_bp = generate_bp(n, True)
```

```

read_once_bp = generate_bp(n, False)

read_twice_monomials = compute_monomials(read_twice_bp)
read_once_monomials = compute_monomials(read_once_bp)

read_twice_deg = hilbert_function(read_twice_monomials)
read_once_deg = hilbert_function(read_once_monomials)

results.append({
    "n": n,
    "read_twice_deg": read_twice_deg,
    "read_once_deg": read_once_deg
})

metric_value_read_twice = sum(result["read_twice_deg"] for result in results) / len(results)
metric_value_read_once = sum(result["read_once_deg"] for result in results) / len(results)

conjecture_holds = all(result["read_twice_deg"] >= n / 2 for result in results) and \
    all(result["read_once_deg"] <= math.log(n, 2) + 1 for result in results)

counterexample = ""
if not conjecture_holds:
    for i, result in enumerate(results):
        if result["read_twice_deg"] < n / 2 or result["read_once_deg"] > math.log(n, 2) + 1:
            counterexample = f"n={result['n']}, read-twice deg={result['read_twice_deg']}, read-once deg={result['read_once_deg']}"
            break

return {
    "metric_name": "Hilbert Function Leading Coefficient",
    "metric_value_read_twice": metric_value_read_twice,
    "metric_value_read_once": metric_value_read_once,
    "instances_tested": len(results),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 35)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_read_twice = sum(result["metric_value_read_twice"] for result in results) / len(results)
    std_read_twice = math.sqrt(sum((result["metric_value_read_twice"] - mean_read_twice)**2 for result in results) / len(results))
    mean_read_once = sum(result["metric_value_read_once"] for result in results) / len(results)
    std_read_once = math.sqrt(sum((result["metric_value_read_once"] - mean_read_once)**2 for result in results) / len(results))

    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean_read_twice={mean_read_twice} std_read_twice={std_read_twice} mean_read_once={mean_read_once} std_read_once={std_read_once}")
    elif any(not result["conjecture_holds"] for result in results) and support_fraction >= 0.8:
        print(f"RESULT: NOT SUPPORTED counterexample={counterexample} support_fraction={support_fraction}")

```

```

    print(f"RESULT: SUPPORTED mean_read_twice={mean_read_twice} std_read_twice={std_read_twice}
else:
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["con]
    print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed=

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_96cab250.py", line 97, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_96cab250.py", line 60, in run_trial
    read_twice_deg = hilbert_function(read_twice_monomials)
                      ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_96cab250.py", line 45, in hilbert_function
    degree = sum(monomial)
              ~~~~~
TypeError: unsupported operand type(s) for +: 'int' and 'str'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to type error in monomial summation, preventing data collection | next:
 Debug monomial processing in hilbert_function to handle type conversions properly

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	84907
2	preregistration	ollama_remote	qwen3:8b	0	0	24230
3	novelty	ollama_remote	qwen3:8b	0	0	20652
4	novelty	ollama_remote	qwen3:8b	0	0	13338
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18753
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11878
7	judge	ollama_remote	qwen3:8b	0	0	15812

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 189571 ms total latency. Provider mix:
 {'ollama_remote': 7}

(full prompt+response transcripts available in research/audit/4febd18902d4.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/4febd18902d4.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/4febd18902d4.tar.gz (if generated)